PROGETTO RETI LOGICHE

Sono un **LAUREANDO** 🎓 (Non so se questo sia il posto giusto dove scriverlo).

> AUTORE: Alessandro Aquilini
> CODICE PERSONA e MATRICOLA: 10891622 & 215881
> PROFESSORE: Fornaciari William

🌐 Questo file è stato scritto con *Notion* e poi esportato in PDF. Per un'esperienza ottimale si consiglia di visualizzarlo online alla pagina corrispettiva: https://superficial-light-8b4.notion.site/RELAZIONE-PROGETTO-RETI-LOGICHE-1efa79e7b9b780ca8ab2c2fbb0ab0e72?pvs=73.

INTRODUZIONE

Obiettivo

🎯 Lo scopo del progetto è implementare un **modulo hardware** descritto in *VHDL* in grado di interfacciarsi con una **memoria esterna** per elaborare dei dati.

In particolare il modulo deve leggere una sequenza di numeri rappresentati in complemento a due a 8 bit (-128 a +127), applicare un **filtro differenziale** di ordine 3 o 5 ed infine scrivere i risultati nella stessa memoria. Il filtro consiste in una **convoluzione** con coefficienti configurabili, seguita da una *normalizzazione* e da una *saturazione* per garantire che l'output rientri nell'intervallo specificato.

L'implementazione deve inoltre gestire correttamente i **segnali di controllo** come START, DONE e RESET.

Specifica

Il modula elabora un messaggio memorizzato in memoria, costituito da:

1. **17 byte di intestazione**:
   - `K1` e `K2` (2 byte): indicano la lunghezza della sequenza di dati da elaborare (K), con K1 come byte più significativo. La lunghezza minima è di 7 byte.
   - `S` (1 byte): determina l'ordine del filtro (3 o 5) in base al bit meno significativo (0 per ordine 3, 1 per ordine 5).

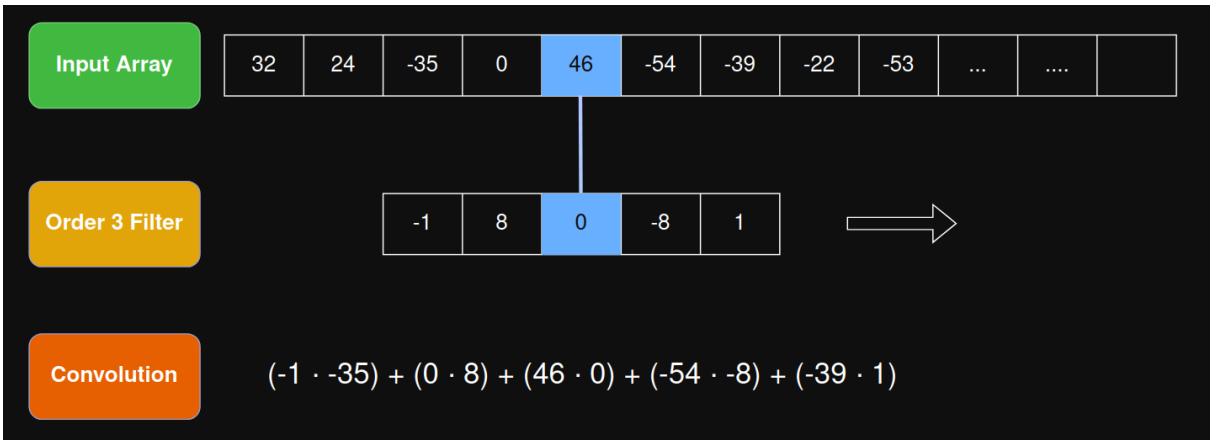
- `C1` a `C14` (14 byte): coefficienti configurabili per i filtri.
  - Ordine 3: `[C2, C3, C4, C5, C6]` con normalizzazione `n=12`.
  - Ordine 5: `[C8, C9, C10, C11, C12, C13, C14]` con normalizzazione `n=60`.

2. **K byte di dati**: valori da -128 a +127 da filtrare.

Il filtro applica la formula:

$$f'(i) = \frac{1}{n} \sum_{j=-I}^{I} C_j \cdot f[j+i]$$

dove $I = 2$ per ordine 3 e $I = 3$ per ordine 5.



Esempio di convoluzione, senza la normalizzazione per 12.

Prendiamo i seguenti dati come *esempio numerico*:

- Coefficients = [0, -1, 8, 0, -8, 1, 0, 1, -9, 45, 0, -45, 9, -1]
- Input Array = [32, -24, -35, 0, 46, -54, -39, -22, -53, ...];

Allora alcuni valori di f saranno:

- $f(0) \text{ con ordine } 3 = \frac{1}{12} \times ([-1, 8, 0, -8, 1] \cdot [\emptyset, \emptyset, 32, -24, -35]) = \frac{1}{12} \times (0 \cdot 32 + -24 \cdot -8 + -35 \cdot 1) = \frac{1}{12} \times 157$
- $f(2) \text{ con ordine } 5 = \frac{1}{60} \times ([1, -9, 45, 0, -45, 9, -1] \cdot [\emptyset, 32, -24, -35, 0, 46, -54]) = \frac{1}{60} \times (1 \cdot 0 + -9 \cdot 32 + \cdots + -1 \cdot -54) = \frac{1}{60} \times (-900)$

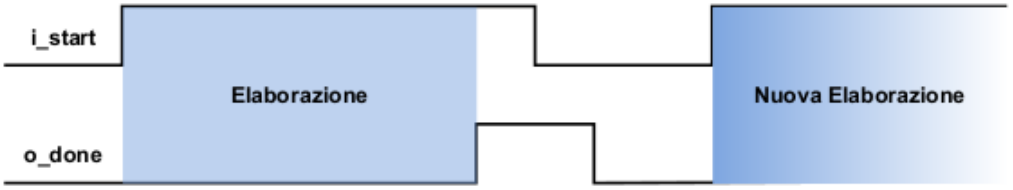
I valori sono poi **normalizzati** e **saturati** nell'intervallo [-128, +127]. Dal momento che la divisione tramite shift a destra causa approssimazioni, si aggiunge +1 al risultato del singolo shift quando sono negativi.

Infine i valori sono scritti in memoria a partire dall'indirizzo `ADD + 17 + K`.

Interfaccia del modulo

```
entity project_reti_logiche is
port (
    i_clk      : in std_logic;
    i_rst      : in std_logic;
    i_start    : in std_logic;
    i_add      : in std_logic_vector(15 downto 0);
    o_done     : out std_logic;
    o_mem_addr : out std_logic_vector(15 downto 0);
    i_mem_data : in std_logic_vector(7 downto 0);
    o_mem_data : out std_logic_vector(7 downto 0);
    o_mem_we   : out std_logic;
    o_mem_en   : out std_logic
);
end project_reti_logiche;
```

- *Segnali di controllo*
  - `i_clk`: segnale di CLOCK generato dal Test Bench, sincronizza tutte le operazioni.
  - `i_rst`: segnale di RESET asincrono; se attivo (`1`), riporta il modulo allo stato iniziale.
  - `i_start`: segnale di START; quando attivo (`1`), avvia l'elaborazione e rimane alto finché `o_done` non viene alzato.
  - `o_done`: segnale di uscita che indica la fine dell'elaborazione; rimane alto finché `i_start` non torna a `0`.
- *Segnali di memoria*
  - `i_add`: indirizzo iniziale a 16 bit fornito dal Test Bench, punta al primo byte dell'intestazione.
  - `o_mem_addr`: indirizzo a 16 bit in uscita verso la memoria, utilizzato per lettura/scrittura.
  - `i_mem_data`: dato a 8 bit in ingresso dalla memoria.
  - `o_mem_data`: dato a 8 bit in uscita verso la memoria, scritto se `o_mem_we` è attivo.
  - `o_mem_en`: segnale di ENABLE per la memoria; se `1`, abilita la comunicazione.
  - `o_mem_we`: segnale di WRITE ENABLE; se `1`, abilita la scrittura altrimenti la lettura.



Protocollo di Elaborazione.

ARCHITETTURA

FSM (Macchina a Stati)

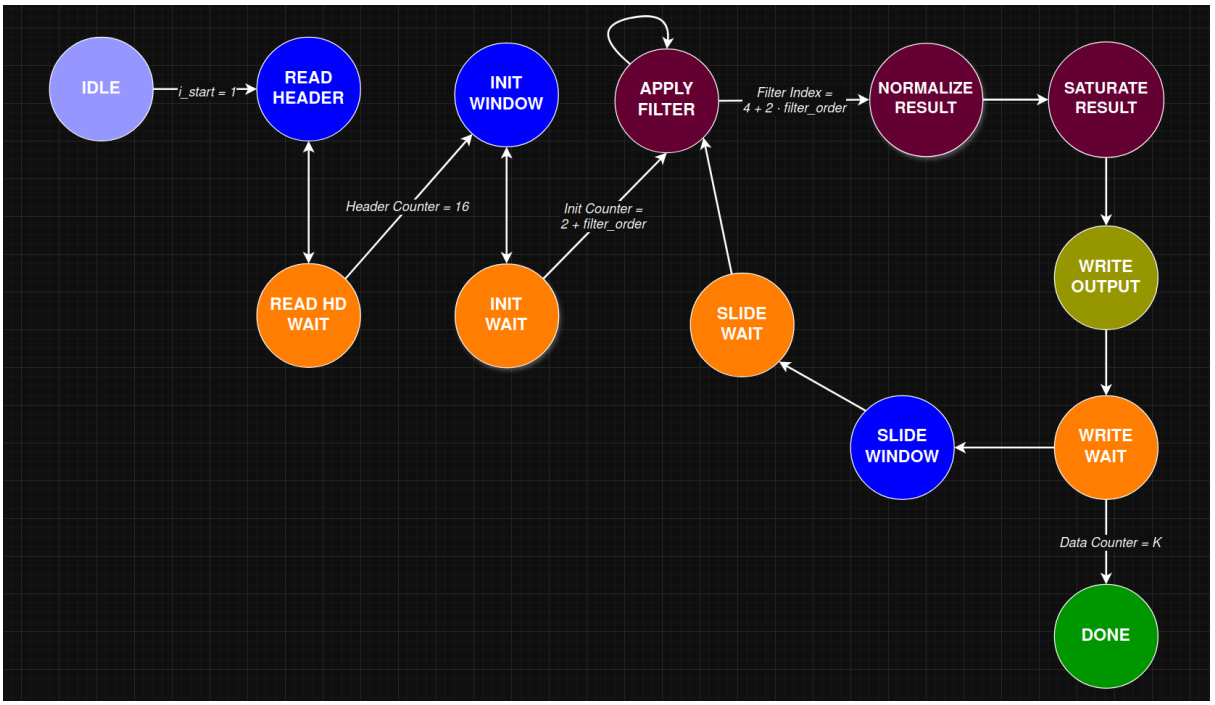


Diagramma della Macchina a Stati.

- 🟢 `IDLE` → É lo **stato iniziale** dove il modulo ritorna ogni volta che `i_rst = 0` e ci rimane fino a quando non c'è una nuova richiesta elaborazione (`i_start = 1`). Tutti i registri interni sono fermi e le uscite verso la memoria sono disabilitate (`o_mem_en = 0`).
- 📥 `READ_HEADER` → Viene impostato l'indirizzo di memoria a partire da `i_add` per leggere i parametri iniziali (K_1, K_2, S e $C_1 \rightarrow C_{14}$). I valori letti da `i_mem_data` vengono caricati progressivamente nei registri interni.
- ⏳ `READ_HD_WAIT` → Poiché la memoria restituisce i dati con **1 ciclo di ritardo**, questo stato serve a sincronizzare la lettura: l'indirizzo è già stato impostato nello stato precedente ed ora il dato effettivo è disponibile.

- 🪟 `INIT_WINDOW` → Viene inizializzata la **finestra di input**, di lunghezza 7. Nel caso di filtro di ordine 3, gli ultimi due valori non vengono mai utilizzati quindi non li acquisiamo nemmeno.
  - Ordine 3 → *[0, 0, input(0), input(1), input(2), -, -]*
  - Ordine 5 → *[0, 0, 0, input(0), input(1), input(2), input(3)]*

  > 0️⃣ Notare che le prime posizioni delle array rimangono a 0, a causa dell'**edge zero padding**.

- ⏳ `INIT_WAIT` → Come nel caso della lettura dell'header, serve un ciclo di attesa per acquisire dati dalla memoria.
- ⚙️ `APPLY_FILTER` → È il nocciolo della progetto in cui ogni elemento della finestra `input_slide` viene moltiplicato per il corrispettivo coefficiente. Viene utilizzato `sum` per accumulare i valori intermedi.

```
when APPLY_FILTER ⇒
   if filter_counter = 0 then
      sum := (others ⇒ '0');
   end if;

   if S(0) = '0' then -- Order 3 (5 coefficients: C2-C6)
      coeff_index := filter_counter + 1;
   else -- Order 5 (7 coefficients: C8-C14)
      coeff_index := filter_counter + 7;
   end if;

   coeff := signed(coefficients(coeff_index*8+7 downto coeff_index*8));

   sum := sum + coeff * signed(input_slide(filter_counter));

   if filter_counter < (4 + 2 * filter_order) then
      filter_counter <= filter_counter + 1;
   else
      filtered_value <= sum;
      filter_counter <= 0;
   end if;
```

- 📊 `NORMALIZE_RESULT` → Il valore calcolato viene diviso in base all'ordine del filtro.

  ▼ Logica per la normalizzazione per 12.

```
when NORMALIZE_RESULT ⇒
   if S(0) = '0' then
      normalized := shift_right(filtered_value, 4) +
                    shift_right(filtered_value, 6) +
                    shift_right(filtered_value, 8) +
                    shift_right(filtered_value, 10);
      -- NOTE: Correct + 4 for vhdl rounding
      if normalized < 0 then
         normalized := normalized + 4;
      end if;
```

- 🚦 `SATURATE_RESULT` → Se dopo la normalizzazione il risultato dovesse uscire dai limiti ammessi, qui viene **saturato**:
  - Valori troppo bassi vengono portati a -127.
  - Valori troppo alti vengono portati a 128.
- 💾 `WRITE_OUTPUT` → Il valore finale (filtrato, normalizzato e saturato) viene **scritto** su `o_mem_data`. In parallelo, `o_mem_addr` viene impostato a `std_logic_vector(unsigned(base_address) + 17 + K + data_counter)`
  e `o_mem_we = 1` abilita la scrittura.
- ⏳ `WRITE_WAIT` → Come già visto per le letture, anche la scrittura avviene al **fronte successivo del clock**.
- ⏭️ `SLIDE_WINDOW` → La finestra di input viene fatta "scorrere" in avanti.
  - Il primo elemento viene scartato.
  - Gli altri valori vengono assegnati all'indice precedente.
  - Il nuovo dato viene letto dalla memoria e inserito nell'ultimo posto della finestra (a meno che non siamo oltre la fine dell'input, e allora settiamo a 0).
- 🏁 `DONE_STATE` → Quando non ci sono più dati da elaborare, il sistema porta `o_done = 1`. L'elaborazione è terminata e si disattiva la memoria `o_mem_en <= '0'` in attesa del segnale di start.

PROCESSI

Per la realizzazione della FSM è stato seguito un approccio **behavioral**, suddividendo il progetto in tre *processi distinti*:

1) Registro di stato: `state_reg`

Si tratta di un **processo sequenziale**, perché è sincronizzato con il clock (`i_clk`). Il suo ruolo è quello di **memorizzare lo stato corrente (`current_state`)**.

Gestisce il *reset asincrono*. Infatti quando `i_rst` è a '1', indipendentemente dal clock, `current_state` viene immediatamente forzato a `IDLE`. Altrimenti, se `i_rst` non è attivo al **fronte di salita del clock** (`rising_edge(i_clk)`), `current_state` assume il valore di `next_state`.

```
state_reg : process(i_clk, i_rst)
    -- Variables
    variable sum : signed(18 downto 0);

begin
    if i_rst = '1' then
        current_state <= IDLE;

        -- Reset registri interni
        base_address <= (others => '0');
        header_counter <= (others => '0');
        init_counter <= (others => '0');

        -- ...

    elsif rising_edge(i_clk) then
        current_state <= next_state;

        case current_state is
            when IDLE =>
                if i_start = '1' then
                    base_address <= i_add;
                    header_counter <= (others => '0');
                end if;

            when READ_HEADER =>
                null;

            -- ...

    end if;
end process;
```

👉 È la parte **sequenziale** che fornisce la memoria della macchina a stati.

2) Funzione di transizione: `next_state_logic`

È un **processo combinatorio**, che serve a determinare il prossimo stato (`next_state`) in funzione dello stato attuale e degli ingressi (ad esempio `i_start`, contatori, ecc.).

```
next_state_logic : process(current_state, i_start, ...)
begin
   case current_state is
      when IDLE ⇒
         if i_start = '1' then
            next_state <= READ_HEADER;
         else
            next_state <= IDLE;
         end if;
      when READ_HEADER ⇒
         next_state <= READ_HD_WAIT;
            when READ_HD_WAIT ⇒
         if header_counter = 16 then
            next_state <= INIT_WINDOW;
         else
            next_state <= READ_HEADER;
         end if;
      ...
   end case;
end process;
```

👉 Qui si implementa la **logica di transizione** della FSM.

3) Logica d'uscita: `output_logic`

È anch'esso un **processo combinatorio** che si occupa di generare tutti i **segnali di uscita** in base allo stato corrente (`current_state`). Qui vengono gestiti i segnali verso la memoria (`o_mem_en`, `o_mem_we`, `o_mem_addr`, `o_mem_data`) e il segnale di completamento (`o_done`).

```
output_logic : process(current_state, ... )
begin
   -- default values
   o_mem_en   <= '0';
   o_mem_we   <= '0';
   o_done     <= '0';
   ...
   case current_state is
      when IDLE ⇒
         o_done <= '0';
      when READ_HEADER ⇒
         o_mem_en <= '1';
      when WRITE_OUTPUT ⇒
```

```
            o_mem_en   <= '1';
            o_mem_we   <= '1';
            o_mem_data <= result;
        when DONE_STATE ⇒
            o_done <= '1';
        ...
    end case;
end process;
```

👉 È la **parte combinatoria delle uscite** (macchina di Moore).

REGISTRI INTERNI

Sono memorizzati in **hardware fisico** e servono per rappresentare lo **stato interno del sistema**.

REGISTRI della FSM

- `current_state` e `next_state`: Sono di tipo `state_type` che implementano la FSM. Rappresentano rispettivamente lo stato corrente e il prossimo stato.

REGISTRI per L'HEADER

- `K` (16 bit unsigned): Memorizza la lunghezza dell'input.
- `S` (8 bit): Il bit meno significativo determina l'ordine del filtro (0 = ordine 3, 1 = ordine 5).
    - `filter_order` (1 bit): Intero da `S(0)` per abbreviare la logica fra ordine 3 e ordine 5.
- `coefficients` (112 bit): Registro che memorizza i 14 coefficienti del filtro (C1-C14). Si potevano risparmiare 2 byte, dal momento che C1 e C7 non sono mai utilizzati.

REGISTRI PER LA GESTIONI DATI

- `input_slide`: Array di 7 registri da 8 bit ciascuno che implementa la finestra scorrevole dell'input.
- `base_address` (16 bit): Memorizza l'indirizzo base di partenza. Man mano che si legge l'header e scrive l'output viene incrementato.

CONTATORI

- `header_counter` (5 bit): Contatore per tracciare i byte dell'header letti dalla memoria (da 0 a 16 byte).
- `init_counter` (3 bit): Utilizzato durante l'inizializzazione della finestra scorrevole.
- `data_counter` (16 bit): Tiene traccia del numero di campioni elaborati (da 0 a K).
- `filter_counter` (intero 0-6): Contatore per l'iterazione attraverso i coefficienti durante l'operazione di filtraggio.

REGISTRI di ELABORAZIONE

- `filtered_value` (19 bit signed): Registro che accumula il risultato della convoluzione prima della normalizzazione.
- `temp_result` (19 bit signed): Registro intermedio che contiene il valore normalizzato.
- `data_buffer` (8 bit): Buffer di output che contiene il valore finale dopo saturazione, pronto per essere scritto in memoria.

RISULTATI SPERIMENTALI

SINTESI

L'utilizzo delle risorse hardware della FPGA, si può verificare attraverso il comando `report_utilization`.

Risorsa	Utilizzate	Disponibili	Utilizzo %
Slice LUTs	496	134.600	0,37%
- LUT come Logica	496	134.600	0,37%
- LUT come Memoria	0	46.200	0,00%
Slice Register	303	269.200	0,11%
- Register come Flip Flop	303	269.200	0,11%
- Register come Latch	0	269.200	0,00%
F7 Muxes	6	67.300	<0,01%
F8 Muxes	0	33.650	0,00%

I risultati evidenziano un paio di aspetti positivi dell'implementazione:

- Non sono stati sintetizzati *latch* (Register as Latch = 0). Ciò garantisce un comportamento temporale prevedibile e evitando potenziali problemi di glitch.
- Tutti i 303 registri sono implementati come *Flip Flop*, approccio ideale per la sincronizzazione con il clock.
- L'implementazione utilizza meno dell'1% di tutte le risorse disponibili.

Il requisito temporale del clock, che ha periodo di 20.000 ns e Duty Cycle del 50% è soddisfatto. Infatti, il comando `report_timing` restituisce un $\Delta = \text{required time} - \text{arrival time} = 9.169 \text{ ns}$.

Parametro	Valore	Stato
Slack	9.169 ns	✅ **MET**

ANALISI dei RITARDI

Tipo Ritardo	Valore	Percentuale
Data Path Delay Totale	10.405 ns	100%
Logic Delay	3.901 ns	37.5%
Routing Delay	6.504 ns	62.5%

ANALISI INCERTEZZA del CLOCK

Parametro	Valore
Total Uncertainty	0.035 ns
Total System Jitter (TSJ)	0.071 ns
Total Input Jitter (TIJ)	0.000 ns
Discrete Jitter (DJ)	0.000 ns
Phase Error (PE)	0.000 ns

SIMULAZIONI

Per verificare il corretto funzionamento del modulo sono stati implementati e eseguiti test che coprono tutti i corner case significativi. Tutti i test sono stati validati sia in *behavioral simulation* che in *post-synthesis functional simulation*, garantendo la correttezza del design a tutti i livelli di astrazione.

Testbench Generator

https://www.bia3ia.com/testbench-generator/

1) Normal Case - Order 3 Filter ✅

Verifica del funzionamento standard per filtro di ordine 3.

```
K = 24
S = 0 (Order 3)
Coefficients = (0, -1, 8, 0, -8, 1, 0, 1, -9, 45, 0, -45, 9, -1)
Base Address = 1234
Input = [32, -24, -35, 0, 46, -54, -39, -22, ..., -62]
```

2) Normal Case - Order 5 Filter ✅

Verifica del funzionamento standard per filtro di ordine 5.

```
K = 17
S = 1 (Order 5)
Coefficients = (0, -1, 8, 0, -8, 1, 0, 1, -9, 45, 0, -45, 9, -1)
```

```
Base Address =  44
Input = [38, 20, -104, 76, 40, 126, 17, 112, -114, -70, -77, 31, 88, -26, -4, 84, 73]
```

3) Edge Case - Minimum Input Length ✅

Test della lunghezza minima dell'input ($K = 7$) e verifica che $C1$ e $C7 \neq 0$ non influenzino il risultato.

```
K = 7 (lunghezza minima specificata)
S = 0 (Order 3)
Coefficients = [105, ... ] (Random)
Input = [-111, -109, 98, 125, -22, 116, -75]
```

4) Edge Case — Saturation & Extreme Values ✅

Test dei valori estremi per verificare l'assenza di overflow e il corretto clamping

```
K = 14
S = 1 (Order 5)
Coefficients = [127, 127, 127, 127, 127, 127, 127, -128, -128, -128, -128, -128, -128, -128]
Input = [127, 127, 127, 127, 127, 127, 127, -128, -128, -128, -128, -128, -128, -128]
```

Il valore più basso che si può ottenere in output è $-128 \cdot 127 \cdot 7 = -113792 > -2^{17}$, mentre il più alto $-128 \cdot -128 \cdot 7 < 2^{17}$. Questa combinazione di *coefficienti* e *input* li va a generare entrambi, testando così il dimensionamento appropriato dei registri interni.

5) Stress Test - Large Input Size ✅

Test di performance con input di grandi dimensioni e verifica della gestione di divisioni negative

```
K = 500
S = 0 (Order 3)
Input = [-14, -17, -118, -24, 27, 81, 76, ..., 106, 48, 19, -67] (500 valori random)
```

6) Memory Boundary Test ✅

Test dei limiti della memoria con massimo utilizzo dello spazio disponibile.

```
K = (2^16 - 17) / 2 (massimo utilizzo memoria)
S = 1 (Order 5)
Base Address = 0
```

CONCLUSIONI

La complessità spaziale del modulo è $O(1)$, poiché vengono allocati registri di dimensioni fisse, indipendenti dalla lunghezza dell'input.
La complessità temporale è invece $O(n)$, dove n è la lunghezza del segnale in ingresso, in quanto è necessario scorrere applicare il filtro su tutte le posizioni della memoria.

Sebbene esistano diversi aspetti che si possono ottimizzare, le complessità ottenute sono le migliori possibili per questo tipo di elaborazione. Inoltre, a mio avviso, il codice che è uscito risulta abbastanza intuitivo.

Ammetto di aver avuto non poche difficoltà con *Vivado* a partire dalla sua installazione. Ho faticato ad orientarmi fra console, impostazioni, file da caricare e messaggi di errore. In particolare il debugging, con la schermata delle onde, è stata un esperienza traumatica. Sarà che non sono troppo familiare con questo tipo di strumenti.

In ogni caso la situazione è migliorata notevolmente dopo aver scritto una *funzione di debug* per stampare messaggi in console durante la simulazione, alla C insomma.

Nonostante le sfide, sono soddisfatto del lavoro svolto. Ho imparato qualcosa di nuovo uscendo dalla mia zona di comfort, e alla fine la soddisfazione di aver capito, almeno a livello generale, come funzionano le cose è inestimabile.